

PAI-2. BYODSEC-BRING YOUR OWN DEVICE SEGURO USANDO ROAD WARRIOR VPN SSL PARA UNA UNIVERSIDAD PÚBLICA

Introducción

En este **Proyecto de Aseguramiento de la Información** se abordará el diseño e implementación de mecanismos que permitan garantizar la **confidencialidad, integridad, autenticidad y disponibilidad de la información** cuando ésta es transmitida a través de redes públicas como Internet.

La **Universidad Pública U-Secure** ha impulsado en los últimos años un proceso de transformación digital que ha incrementado significativamente el acceso remoto a sus servicios institucionales. Actualmente, el personal docente, investigador y de administración necesita acceder a recursos corporativos desde diferentes ubicaciones, incluyendo sus domicilios, congresos, estancias de investigación o periodos de teletrabajo.

Para favorecer la movilidad y flexibilidad de su comunidad universitaria, la organización ha adoptado una política de **Bring Your Own Device (BYOD)**, permitiendo que los empleados utilicen dispositivos personales como ordenadores portátiles, tabletas o teléfonos móviles para acceder a servicios institucionales.

Entre los recursos accesibles de forma remota se encuentran:

- Correo electrónico corporativo.
- Plataformas académicas y de docencia virtual.
- Sistemas de gestión administrativa.
- Repositorios documentales.
- Bases de datos de investigación.
- Servidores internos y aplicaciones corporativas.

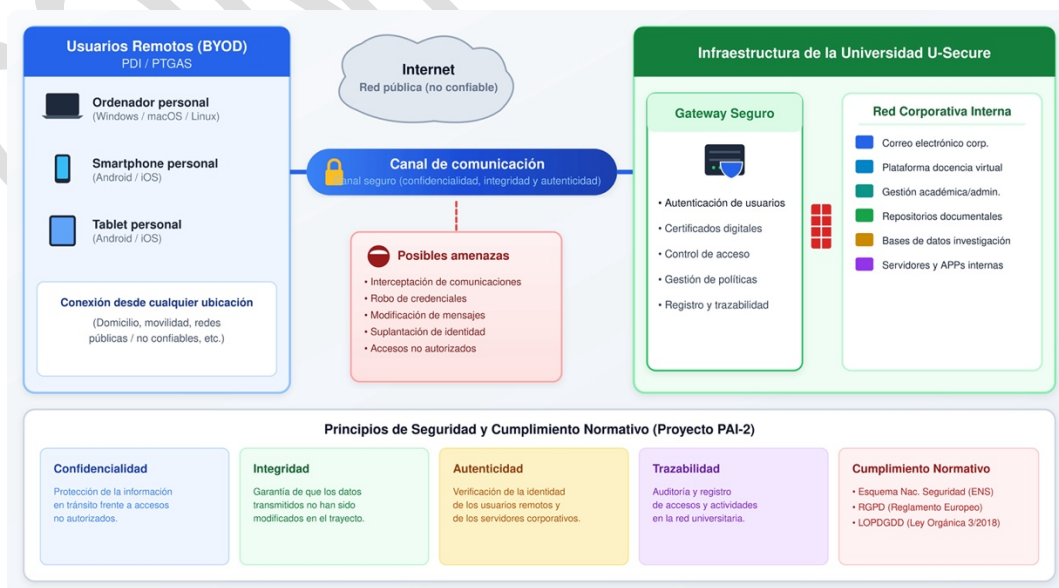


Figura 1: Infraestructura de conexión BYOD para Universidad Pública U-Secure

Sin embargo, este modelo plantea importantes desafíos de seguridad. Los usuarios pueden conectarse desde redes domésticas, redes públicas o infraestructuras fuera del control de la organización, exponiendo la información a riesgos como:

- Interceptación de comunicaciones.
- Robo de credenciales.
- Accesos no autorizados.
- Manipulación de información en tránsito.
- Suplantación de identidad.
- Compromiso de dispositivos remotos.

Como respuesta a estos riesgos, la Universidad ha decidido desplegar una infraestructura de **acceso remoto seguro** basada en tecnologías VPN y protocolos criptográficos que permitan establecer canales de comunicación protegidos entre los usuarios remotos y la infraestructura corporativa.

Para ello será necesario implementar mecanismos que proporcionen:

- Autenticación de usuarios y servidores.
- Confidencialidad de la información transmitida.
- Integridad de los datos intercambiados.
- Protección frente a ataques de interceptación y manipulación.
- Gestión segura de credenciales y certificados digitales.
- Registro y trazabilidad de los accesos realizados.

Las tecnologías empleadas deberán apoyarse en protocolos seguros como **TLS**, infraestructuras de clave pública (**PKI**), certificados digitales y mecanismos de autenticación robusta, reproduciendo los principios utilizados por soluciones profesionales de acceso remoto seguro.

Además de los requisitos técnicos, la solución deberá considerar el marco normativo aplicable a las Administraciones Públicas españolas. La Universidad debe garantizar el cumplimiento del **Reglamento General de Protección de Datos (RGPD)**, de la **Ley Orgánica 3/2018 de Protección de Datos Personales y Garantía de los Derechos Digitales (LOPDGDD)** y del **Esquema Nacional de Seguridad (ENS)**, que establece los principios y requisitos mínimos para proteger adecuadamente la información y los servicios gestionados por medios electrónicos.

Política de Seguridad

La política de seguridad de la **Universidad Pública U-Secure** nos dice con respecto a la seguridad en las transmisiones de información cliente-servidor:

“Todas las comunicaciones realizadas entre usuarios remotos y los sistemas corporativos de la Universidad Pública U-Secure deberán establecerse a través de canales de comunicación seguros que garanticen la confidencialidad, integridad y autenticidad de la información transmitida. La organización deberá garantizar que únicamente usuarios, dispositivos y servicios previamente autorizados puedan acceder a los recursos corporativos mediante mecanismos robustos de autenticación y protección de las comunicaciones frente a amenazas como la interceptación de tráfico o la suplantación de identidad.”

Objetivos

1. Desarrollar/seleccionar **cómo llevar a la práctica de forma lo más eficiente posible los canales de comunicación segura para la transmisión de credenciales (usuario, contraseñas) y mensajes usando** el protocolo TLS asegurando autenticidad, confidencialidad e integridad.

2. **Implementar una Infraestructura de Clave Pública (PKI) y canal seguro basado en TLS 1.3** con Cipher Suites robustas, garantizando la confidencialidad, integridad y autenticación mutua (mTLS) en el intercambio de credenciales y datos.
3. **Evaluar el rendimiento, sobrecoste (overhead) y escalabilidad del túnel VPN** para garantizar el soporte de una carga de trabajo estimada de **300 empleados concurrentes**, comparando cuantitativamente el comportamiento del tráfico cifrado frente al tráfico no encapsulado.
4. **Verificar y auditar la seguridad mediante análisis de tráfico y simulación de ataques**, generando evidencias técnicas (archivos .pcap y registros de auditoría).
5. **Extra opcional:** Demostrar de forma práctica en un entorno controlado la capacidad de la arquitectura VPN TLS (mTLS + TLS 1.3) para resistir e identificar ataques de interceptación y suplantación en redes públicas o no confiables (+10%)

Recomendaciones

Arquitectura

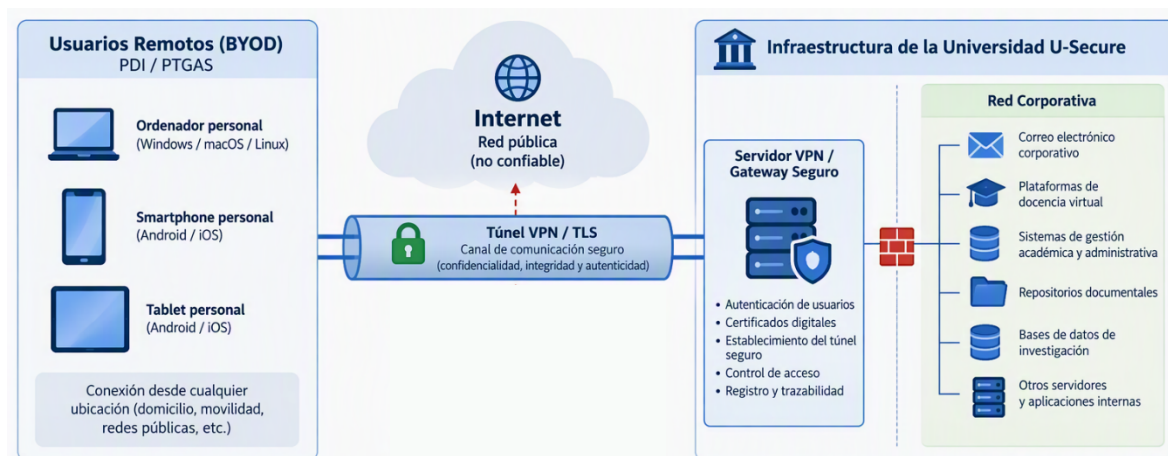


Figura 2: Escenario de canal de comunicación seguro usando VPN TLS

Requisitos funcionales

RF1. Registro de Usuarios y Emparejamiento de Dispositivos (PKI / BYOD)

- **Registro de Credenciales y Certificado:** Permitir el registro de un nuevo usuario proporcionando un nombre de usuario y contraseña corporativa, vinculándolo al proceso de emisión/solicitud de un certificado.
- **Inmutabilidad de Registros:** Una vez completado el registro, los datos de identidad y la clave pública asociada no podrán ser modificados sin un proceso explícito de re-emisión y revocación.

RF2. Inicio de Sesión y Autenticación Mutua (mTLS + Credenciales)

- **Autenticación en Dos Capas (mTLS):** Permitir el inicio de sesión únicamente si se supera la validación previa del certificado digital del dispositivo BYOD (capa de red/transporte) combinada con el par usuario/contraseña introducida por el usuario (capa de aplicación).

RF4. Cierre Seguro de Sesión y Desmontaje del Túnel

- **Finalización de Canal:** Permitir al usuario cerrar la sesión de forma explícita, forzando la destrucción de las claves efímeras de sesión (PFS) y el desmontaje inmediato del túnel seguro.

RF5. Gestión de Usuarios Preexistentes y Perfiles de Acceso

- **Precarga de Usuarios Corporativos:** El sistema debe contar con un conjunto inicial de cuentas precargadas pertenecientes a los roles de la universidad (PDI, PAS e Investigadores) con sus correspondientes certificados emitidos para habilitar las pruebas de carga y concurrencia.

Capa de transporte

Tecnología	Evolución en PAI-2
Sockets TCP	Sockets TLS 1.3 Nativos: Uso de librerías criptográficas nativas (ssl en Python con OpenSSL, JSSE en Java, OpenSSL en C/C++) configurando mTLS con validación de certificado X.509 de cliente en el <i>handshake</i> .
API REST	HTTPS con Autenticación Mutua (mTLS): Servidor web/API sobre HTTPS restringido a TLS 1.3, configurado para exigir certificado de cliente (verify_client / ssl_verify_mode) en cada petición REST.

gRPC	gRPC sobre TLS 1.3 con Channel Credentials (mTLS): Configuración de canales seguros en gRPC (grpc.ssl_channel_credentials / grpc.ssl_server_credentials) exigiendo mTLS en el multiplexado HTTP/2.
------	--

Requisitos de seguridad

RS1. Infraestructura PKI y Autenticación Mutua

- **Autoridad de Certificación (CA) Corporativa:** Generación y gestión del ciclo de vida de certificados digitales X.509 para el servidor VPN U-Secure y los dispositivos remotos (Ver anexos)
- **Autenticación Mutua (mTLS):** Validación bidireccional estricta donde el cliente valida el certificado del servidor corporativo y el servidor exige/comprueba el certificado digital del dispositivo BYOD antes de establecer la conexión.

RS2. Canal Seguro y Selección de Cipher Suites (TLS 1.3)

- **Protocolo TLS 1.3:** Restricción estricta de versiones obsoletas o vulnerables (SSLv3, TLS 1.0, TLS 1.1 y TLS 1.2 descatalogados).
- **Suites Criptográficas Robustas:** Configuración exclusiva de algoritmos con *Perfect Forward Secrecy* (PFS) y cifrado autenticado (AEAD), tales como TLS_AES_256_GCM_SHA384 o TLS_CHACHA20_POLY1305_SHA256.

RS3. Escalabilidad, Capacidad y Benchmarking (300 Usuarios)

- **Modelo de Concurrencia:** Diseño de la arquitectura de red y tunelización preparado para gestionar un volumen de **300 sesiones simultáneas**.
- **Análisis de Rendimiento Comparativo:** Evaluación experimental y cuantitativa de la pérdida de rendimiento (*throughput*, latencia/RTT y/o consumo de CPU/RAM) del tráfico a través de la VPN TLS frente a conexiones directas sin cifrar.

RS4. Auditoría, Análisis de Tráfico y Trazabilidad

- **Inspección de Paquetes (.pcap):** Generación de capturas de red en Wireshark/tcpdump que demuestren que las credenciales y datos sensibles viajan 100% cifrados e ininteligibles dentro del túnel sobre la interfaz pública.
- **Registros de Auditoría:** Registro trazable de eventos de conexión, desconexión e intentos de acceso no autorizados.

Normas del entregable

- Cada grupo debe entregar a través de la Plataforma de Enseñanza Virtual y en la **actividad** preparada para ello un archivo zip, nombrado **PAI2-STX.zip**, que deberá contener al menos los ficheros siguientes:
 - ✓ **Código fuente de las posibles implementaciones y/o scripts desarrollados y/o configuraciones, será necesario la entrega de los test, logs de las pruebas y trazas de los sniffers utilizados.**
 - ✓ **Documento PDF (máximo 15 páginas), estructura del documento:**
 - **Resumen:** indicar brevemente las decisiones técnicas y algorítmicas implementadas para satisfacer los requisitos funcionales y de seguridad.
 - **Manual de despliegue y uso:** instrucciones de instalación y puesta en funcionamiento del proyecto, y se explique y detalle las funcionalidades principales del proyecto.
 - **Grado de completitud:** indicar de manera trazable el grado de cumplimiento de los objetivos, requisitos funcionales y de seguridad frente al código entregado.
- El plazo de entrega de dicho proyecto finaliza el **día 27 de octubre a las 23:59 horas**.

- Los proyectos entregados fuera del plazo establecidos serán considerados inadecuados por el cliente y por tanto entrarán en penalización por cada día de retraso entrega de 10% del total, hasta agotarse los puntos.
- **El cliente no aceptará envíos realizados por email, ni mensajes internos de la enseñanza virtual, ni correo interno de la enseñanza virtual. Toda entrega realizada por estos medios conllevará una penalización en la entrega del 10%.**

CONFIDENCIAL

ANEXOS

ANEXO I - Java de infraestructura cliente-servidor TLS con Sockets

Arquitectura Cliente-Servidor

En primer lugar, vamos a crear una infraestructura cliente-servidor mediante el uso de sockets seguros.

A. Creación de un keyStore – Compartición de claves

De acuerdo con la especificación del protocolo SSL/TLS se necesita para la autenticación de los servidores de los correspondientes certificados. En Java se puede realizar esto mediante la creación de un almacén o repositorio de certificados de seguridad para el protocolo SSL/TLS. Java viene provisto de una herramienta que permite crear de manera sencilla un almacén de certificados. Para crear el almacén de certificados ejecutamos en consola y con permisos de administración el siguiente comando:

```
keytool -genkey -keystore k1.jks -alias ssl -keyalg RSA
```

Keytool hará una serie de preguntas para crear la creación del keyStore. Con esto ya tendremos en el path donde corramos este comando, generando un archivo, por ejemplo, con nombre **k1**. Este será el almacén de certificados que usaran nuestros Sockets cliente y servidor. De manera similar se podría realizar en un sistema basado en Linux.

Para usuarios de Windows, **keytool** se encuentra en la carpeta *bin* del JDK ¹instalado en el sistema. Por lo tanto, para usar **keytool** desde línea de comandos sin tener que entrar en la ruta debemos configurar las variables de entorno JAVA_HOME y PATH con la ruta hacia la carpeta *bin* de la JDK de Java.

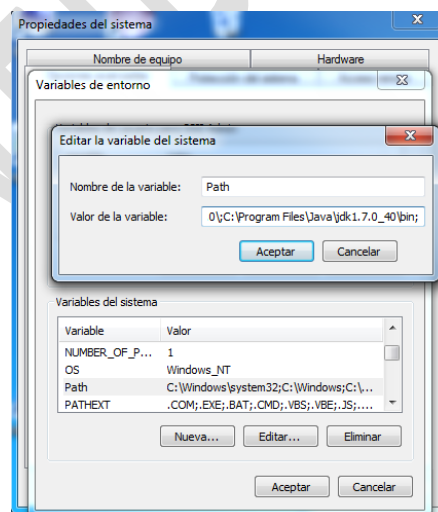


Figura 3: Configuración variable de entorno

B. Generación de un Socket TLS Servidor

Para utilizar comunicaciones seguras usando SSL/TLS entre cliente-servidor, utilizaremos Socket SSL/TLS. Crearemos una clase, *por ejemplo*, **EjemploSocketSSLServer.java**, donde instanciaremos el server socket, pero usando la implementación SSL/TLS (**SSLServerSocket**):

¹ Cuidado porque los JRE de java no proporcionan dicha herramienta, sólo los JDK de Oracle.

```
import javax.net.ssl.*; // Librerías SSL/TLS de JDK

...

SSLServerSocketFactory socketFactory = (SSLServerSocketFactory)
    SSLServerSocketFactory.getDefault();
serverSocket = (SSLServerSocket) socketFactory.createServerSocket(3343);

SSLSocket socket = (SSLSocket) serverSocket.accept();
```

El código se puede ejecutar en cualquier IDE de desarrollo (Eclipse o Visual Studio Code), pero debemos tener en cuenta que son dos procesos (servidor y cliente) diferentes con diferentes parámetros, por tanto, deberemos tener activada la correspondiente configuración de entorno. O también podemos no ejecutar directamente este código sobre el IDE, y hacerlo en el entorno de ejecución del sistema operativo especificando los procesos y los parámetros a la JVM de dónde se encuentra el almacén de claves y la clave de este como veremos en el apartado D.

C. Generación de un Socket TLS Cliente

Crearemos una clase **EjemploSocketSSLClient.java** donde crearemos una conexión a un socket server determinado pero usando la implementación SSL/TLS (**SSLSocket**), tal y como se muestra a continuación:

```
import javax.net.ssl.*; // Librerías SSL/TLS de JDK

...

SSLSocketFactory socketFactory = (SSLSocketFactory) SSLSocketFactory.getDefault();
SSLSocket socket = (SSLSocket) socketFactory.createSocket("localhost", 8080);
```

Notar que la IP y el puerto de conexión deben coincidir con el puerto e IP del servidor y el puerto definido en la clase del servidor.

D. Compilar y ejecutar código cliente y servidor desde consola

1. Desde una consola con permisos de administración y en el directorio donde se encuentre la clase Java del cliente o servidor, ejecutaremos el siguiente comando:

```
javac2 EjemploSocketSSLServer.java
javac EjemploSocketSSLCliente.java
```

Esto va a generar un archivo .class con el código interpretable por la JVM de Java, y que utilizaremos desde la misma línea de comandos para su ejecución.

2. Para ejecutar las clases una vez compiladas simplemente debemos de ejecutar el siguiente comando donde se establecen el keyStore en el servidor y el trustStore para el cliente:

- **EjemploSocketSSLServer.java:**

```
java -Djavax.net.ssl.keyStore=C:\SSLStore -Djavax.net.ssl.keyStorePassword=PASSWORD
EjemploSocketSSLServer
```

- **EjemploSocketSSLCliente.java:**

² El compilador javac sólo está disponible en las JDK.


```
java -Djavax.net.ssl.trustStore=C:\SSLStore -Djavax.net.ssl.trustStorePassword=PASSWORD  
EjemploSocketSSLCliente
```

ANEXO II – Certificados x.509

A.1. Concepto y Rol de los Certificados Digitales

Un **certificado digital** es un documento electrónico firmado criptográficamente por una Autoridad de Certificación (CA) que vincula una identidad (persona, dispositivo o servicio) con una clave pública. El estándar **ITU-T X.509 (versión 3)** define el formato ASN.1 (ver Figura 3) estructurado utilizado para la representación de certificados de clave pública.



Figura 4: Estructura de un certificado x.509 v3

ANEXO III – TLS muto

mTLS (Mutual TLS o TLS Mutuo) es un método de seguridad en el que ambas partes de la comunicación (cliente y servidor) se autentican mutuamente mediante certificados digitales X.509 antes de establecer el canal cifrado. Es decir, tanto el cliente como el servidor tienen un certificado, y ambas partes se autentican utilizando su par de claves públicas/privadas.

En comparación con el TLS normal, en mTLS hay pasos adicionales para verificar a ambas partes:

1. El cliente se conecta al servidor.
2. El servidor presenta su certificado TLS.
3. El cliente verifica el certificado del servidor.
4. El cliente presenta su certificado TLS.
5. El servidor verifica el certificado del cliente.
6. El servidor concede el acceso.
7. El cliente y el servidor intercambian información a través de una conexión TLS encriptada.

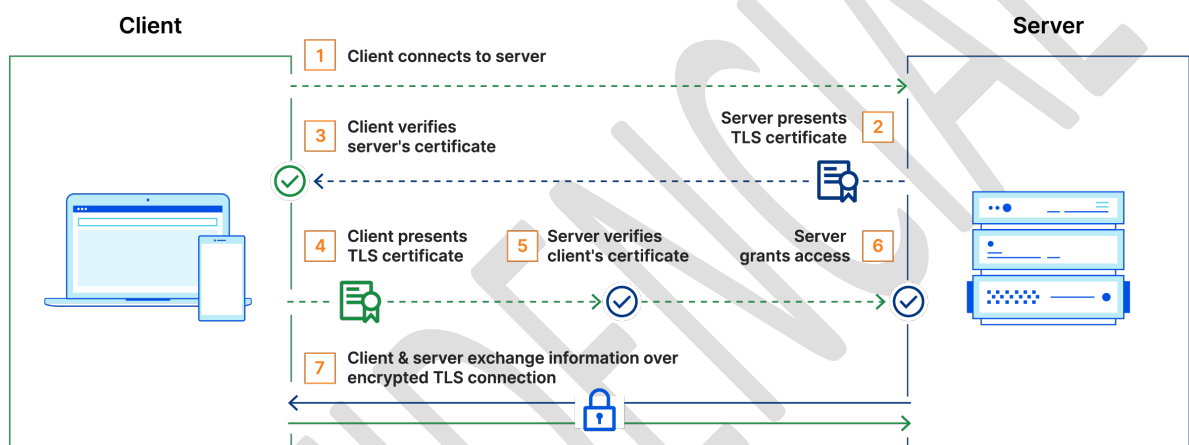


Figura 5: Protocolo de mTLS (fuente: Cloudflare)